Visão Computacional e Aprendizado Profundo

Fundamentos de CNNs, Treinamento YOLOv8, Exportação ONNX e Análise de Resultados

Prof. Me. Sergio Souza Novak — UniRV

Universidade de Rio Verde – UniRV
Pesquisa em Aquicultura e Visão Computacional

21 de setembro de 2026



Sumário

1. Redes Neurais Convolucionais (CNNs)
2. Treinamento vs. Inferência
3. Treinamento de Redes YOLO (YOLOv8)
4. Pasta de Resultados e Exportação ONNX

O que são Redes Neurais Convolucionais (CNNs)?

Definição

Redes Neurais Convolucionais (CNNs ou ConvNets) são arquiteturas de Aprendizado Profundo (*Deep Learning*) especializadas no processamento e análise de dados em formato de grade, como imagens bidimensionais.

Operação de Convolução

- Utiliza **filtros/kernels** (matrizes pequenas de pesos).
- Desliza sobre a imagem calculando produtos escalares locais.
- Gera **mapas de características** (*feature maps*).

Extração Hierárquica

- **Camadas Iniciais:** Extraem bordas, linhas e gradientes de cor.
- **Camadas Intermediárias:** Identificam texturas e formas geométricas.
- **Camadas Profundas:** Reconhecem partes de objetos e peixes completos.

Arquitetura e Camadas Principais de uma CNN

- **Camadas Convolucionais (Conv2D):** Extração de padrões espaciais.
- **Funções de Ativação (SiLU / ReLU):** Não-linearidade para aprender relações complexas.
- **Pooling / Stride:** Redução espacial garantindo invariância a rotações/translações.

Fluxo de Processamento em uma CNN (Pipeline TikZ)



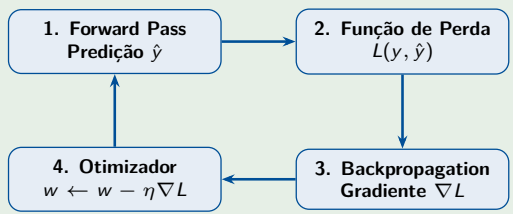
O Processo de Treinamento

Aprendizado Supervisionado

Ajuste iterativo de parâmetros (pesos e viéses) para minimizar o erro entre a predição da rede e o rótulo real (**Ground Truth**).

1. **Forward Pass:** Calcula predição $\hat{y}$.
2. **Loss Function:** Mede o erro $L(y, \hat{y})$.
3. **Backpropagation:** Deriva gradientes ∇L.
4. **Optimizer:** Atualiza pesos ($w \leftarrow w - \eta \nabla L$).

Ciclo de Treinamento (Diagrama TikZ)



Treinamento vs. Inferência: Comparativo

O que é Inferência?

É a fase de uso prático do modelo já treinado. O modelo recebe uma nova imagem e produz predições em tempo real com os **pesos congelados** (*read-only*).

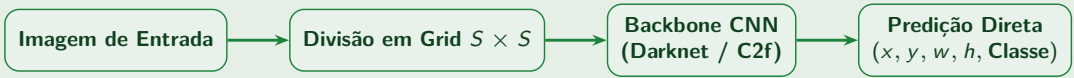
Característica	**Treinamento**	**Inferência**
Objetivo	Ajustar pesos da rede	Fazer predições operacionais
Atualização de Pesos	Sim (Backpropagation)	Não (Pesos congelados)
Uso de Memória/GPU	Muito Alto (armazena gradientes)	Baixo / Moderado
Velocidade	Lenta (várias épocas)	Rápida (milissegundos / FPS real)
Dados de Entrada	Conjunto de Treino + Validação	Novas imagens / Vídeos de campo

Filosofia e Arquitetura do YOLO (YOLOv8)

You Only Look Once

Diferente de detectores em dois estágios (como R-CNN), o YOLO formula a detecção de objetos como um **único problema de regressão**, analisando a imagem inteira em uma única passada.

Fluxo de Detecção em Estágio Único (Single-Stage TikZ)



Inovações do YOLOv8

Design **Anchor-Free** (sem ancoragens pré-definidas) e **Decoupled Head** (separação independente de classificação e localização).

Funções de Perda (Losses) no Treinamento YOLOv8

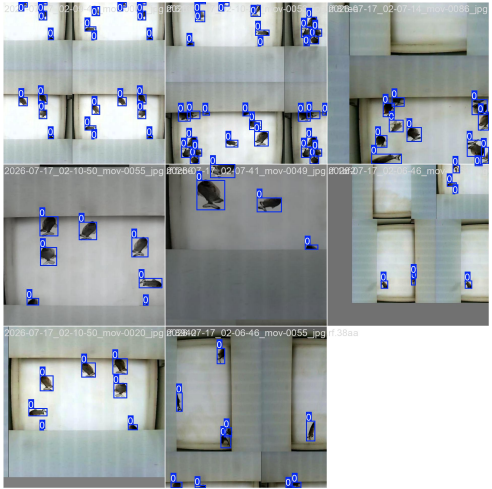
O treinamento do YOLOv8 utiliza uma função de perda composta por três componentes essenciais:

- **Box Loss (CIoU / GIoU Loss):** Mede a sobreposição espacial, o alinhamento central e a proporção de aspecto entre a caixa predita e a caixa real do peixe.
- **Distribution Focal Loss (DFL):** Otimiza a regressão de bordas de forma contínua, permitindo localização milimétrica quando o peixe está ocluso ou em movimento.
- **Classification Loss (BCE Loss):** Utiliza Entropia Cruzada Binária para distinguir com precisão o peixe (*Caranha*) de elementos do fundo.

Perda Total de Treinamento

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{dfl}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}}$$

Aumento de Dados (Data Augmentation) e Hiperparâmetros



Mosaic Augmentation (Exemplo Real)

- Combina 4 imagens aleatórias em um único batch de treino (`train_batch0.jpg`).
- Força a rede a reconhecer peixes em diferentes escalas e posições.

Hiperparâmetros (`args.yaml`)

- **Modelo Base:** YOLOv8s (`yolo8s.pt`)
- **Épocas:** 600 épocas | **Batch:** 8
- **Resolução:** 640×640 pixels
- **Learning Rate (lr_0):** 0.01

Estrutura da Pasta de Resultados (caranha8cm_v8)

O diretório caranha8cm_v8 armazena todo o histórico do experimento realizado para a detecção de alevinos de caranha (8cm):

Arquivos de Configuração e Pesos

- args.yaml: Parâmetros de treino.
- weights/best.pt: Pesos de melhor desempenho.
- weights/last.pt: Pesos da última época (600).

Métricas e Diagnósticos

- results.csv / results.png: Curvas de treino.
- confusion_matrix.png: Matriz de confusão.
- BoxPR_curve.png / BoxF1_curve.png: Curvas de precisão e recall.
- val_batch*_pred.jpg: Predições visuais.

Reprodutibilidade e Pesos: `args.yaml` e `weights/`

O Arquivo `args.yaml`

Garante a **reprodutibilidade científica** do experimento. Contém o registro exato da arquitetura, semente aleatória (`seed=0`), otimizadores e hiperparâmetros de aumento de dados utilizados.

`weights/best.pt`

- Pesos que atingiram o menor valor de perda e maior mAP no conjunto de validação.
- **Uso recomendado:** Inferência final e implantação em produção.

`weights/last.pt`

- Estado exato dos pesos ao final da 600ª época.
- **Uso recomendado:** Retomada do treinamento (`resume=True`).

Exportação do Modelo: Conversão de best.pt para ONNX

Pipeline de Conversão de Modelo (Diagrama TikZ)



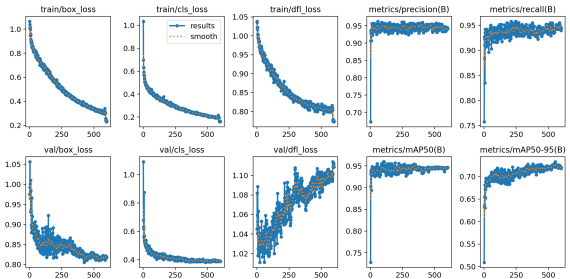
Código de Exportação em Python

```python
from ultralytics import YOLO

# 1. Carrega os melhores pesos PyTorch (.pt)
model = YOLO('weights/best.pt')

# 2. Exporta o modelo para o formato ONNX (.onnx)
model.export(format='onnx', dynamic=True, simplify=True)
```

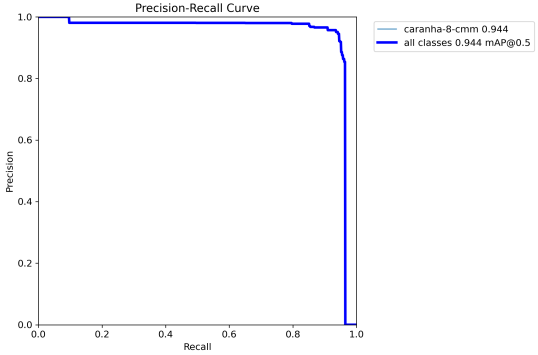
Análise da Convergência: `results.csv` e `results.png`



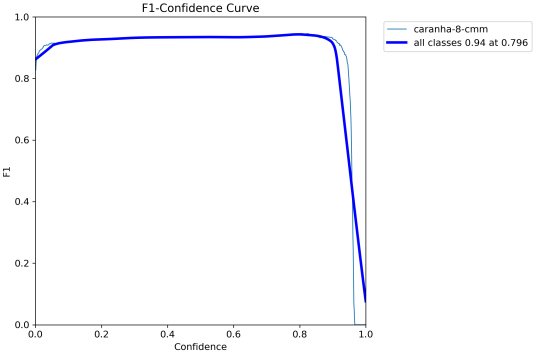
O que o gráfico demonstra?

- **Redução constante de Perdas:** `box_loss`, `cls_loss` e `dfl_loss` diminuem consistentemente.
- **Evolução das Métricas:**
  - **Precisão (B):** $\approx 94.1\%$
  - **Recall (B):** $\approx 94.5\%$
  - **mAP50: 94.6%** (Excelente detecção com IoU 0.50).
  - **mAP50-95: 71.9%** (Localização precisa em rigor elevado).

Curvas de Desempenho (PR e F1-Score)



Curva Precision-Recall (`BoxPR_curve.png`)

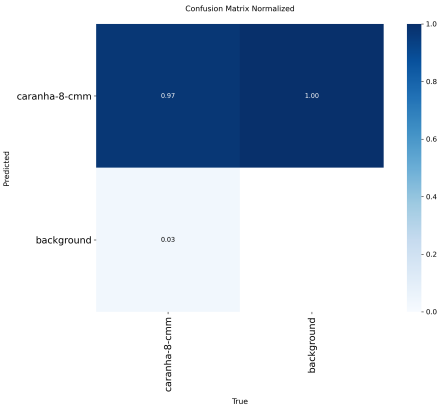


Curva F1-Score vs. Confiança (`BoxF1_curve.png`)

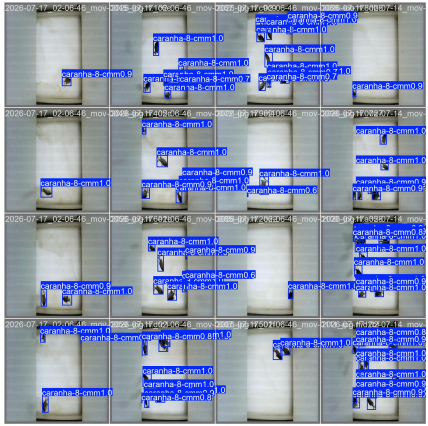
Análise Visual das Curvas

A curva PR confirma alta área sob a curva (mAP50 = 0.946), enquanto a curva F1 indica o limiar ótimo de confiança para a detecção de alevinos sem falsos alarmes.

Matriz de Confusão e Diagnósticos Visuais



Matriz de Confusão Normalizada



Predições no Batch de Validação (`val_batch0_pred.jpg`)

Validação Prática

A matriz normalizada atesta alta acurácia na diagonal principal. As imagens do conjunto de validação confirmam visualmente a precisão das caixas delimitadoras e scores de confiança sobre os alevinos.

Conclusão e Ciclo Completo de Visão Computacional

Resumo Metodológico

1. **Coleta e Anotação:** Imagens de alevinos de caranha (8cm) em ambiente aquático.
2. **Arquitetura CNN/YOLOv8:** Extração de características espaciais e regressão de bounding boxes.
3. **Treinamento Supervisionado:** Otimização iterativa em 600 épocas com aumento Mosaic.
4. **Auditoria de Resultados:** Validação rigorosa via `results.csv`, matriz de confusão e mAP ($\approx 94.6\%$).
5. **Exportação & Inferência:** Conversão do `best.pt` para ONNX e implantação operacional.

Obrigado!

Prof. Me. Sergio Souza Novak — UniRV